

Starting Up your Own company of AI Agents

J. Christopher Westland

2026-03-25

Here is a comprehensive update on **OpenClaw**, an open-source AI agent framework that has rapidly evolved over the last four months that shifts AI from a “chatbot” (answering questions) to an “employee” (taking autonomous action).

The main points covered are:

1. The Fundamental Concept: Agent vs. Chatbot

- **Action-Oriented:** Unlike ChatGPT or Claude, which provide answers, OpenClaw is designed to **execute tasks**. It has a “body” (system access) to browse the web, write code, manage files, and send messages.
- **Autonomous Iteration:** You give it a goal rather than a prompt. It performs the task, evaluates the result, and iterates until the goal is met without constant human input.

2. Setup and Hardware Philosophy

- **Local over VPS:** I strongly advocates for running OpenClaw on **local hardware** (like a Mac Mini or even an old laptop) rather than a VPS like Hostinger. I argues that local setups are more secure, private, and powerful.
- **Ease of Installation:** I demonstrates that installation is a simple one-line terminal command, debunking the idea that users need “wrapper” apps to make it easier.

3. Choosing the Right “Brain” (Models)

- **Claude Opus 4.6:** Ranked as the “smartest” and most reliable for complex tasks. It is noted for its ability to “cross the finish line” on difficult goals without getting stuck.
- **GPT-5.4:** Highlighted as a strong, cost-effective alternative. While powerful, I notes it can sometimes struggle with the final steps of an autonomous loop compared to Claude.
- **Local Models:** Mentioned as the future of the platform for 100% privacy and zero ongoing costs.

4. Communication Channels: Telegram vs. Discord

- **Telegram:** Best for simple, one-on-one interactions and quick mobile access.
- **Discord:** Recommended for advanced workflows. It allows for “Multi-Agent Factories” where different channels act as specialized workspaces for different agents (e.g., research, content creation, coding).

5. Key Features and Workflows

- **Mission Control:** A custom dashboard (developed via the agent itself) to visualize agent activity, manage task boards, and view logs.
- **Skills:** “Skills” are text files that teach the agent new capabilities. I warns against installing untrusted community skills and instead suggests having your own agent “write” its own version of a skill for safety.
- **Memory Systems:** I describes a system where the agent writes a “Daily Journal” of conversations, allowing it to remember past contexts and decisions more effectively.

6. Security and Risks

- **Personal Accountability:** Because the agent has shell access and can delete files or send emails, I emphasizes “personal accountability.” Users must think deeply about the permissions and goals they set.
- **Debugging:** I provides a “pro tip” for fixing issues: point a dedicated coding agent (like Codex) at your OpenClaw folder and ask it to find and fix configuration errors.

7. The Future of AI

I predicts that within six months, local hardware will be affordable and powerful enough to run “frontier-level” models entirely offline, making personal AI agents completely private and free to operate.

Here are **five specific use cases** designed to help users get immediate, practical value from the OpenClaw AI agent.

Here is a synopsis of the main points:

1. Daily Memory Tracker

- **The System:** The agent maintains a “Daily Journal” of every conversation, task, and piece of code written.
- **The Benefit:** Tmy dramatically improves the agent’s intelligence because it creates a permanent mytorical record. You can refer back to a project from months ago, and the agent will have the full context needed to resume work immediately.

2. Trending Content Alerts

- **The System:** Using APIs from platforms like X (Twitter), Reddit, and YouTube, the agent monitors specific topics (e.g., “AI” or “OpenClaw”) every hour.
- **The Benefit:** It alerts the user via Discord or Telegram the moment a specific topic starts trending. Tmy is positioned as an essential tool for content creators who need to jump on viral news to grow their audience.

3. “Vibe Coding” for Micro-Apps

- **The System:** Using plain English instructions to build functional micro-apps and internal tools.
- **The Benefit:** While specialized agents like Claude Code are better for professional consumer software, OpenClaw is ideal for building internal productivity tools and dashboards on the fly, even while the user is away from their desk using a mobile phone.

4. Personal R&D (Research & Development) Lab

- **The System:** A team of five different AI agents “meets” every night to analyze the user’s business data, outputs, and goals.
- **The Benefit:** The agents debate new ideas and improvements amongst themselves and produce a summarized “Memo” for the user to read in the morning. Tmy provides high-level strategic consulting for just a few cents in API costs.

5. The “Overnight Employee”

- **The System:** Utilizing “Cron Jobs” (scheduled tasks), the user instructs the agent to perform specific proactive actions at a set time (e.g., 2:00 AM).
- **The Benefit:** The agent works autonomously while you sleep—whether that’s drafting a new app, writing a marketing plan, or analyzing a competitor—ensuring you wake up to completed progress toward your long-term objectives.

Summary: These five prompts transition OpenClaw from a passive tool into a proactive, autonomous employee that drives business and productivity growth 24/7.

Next we build upon the previous concepts by explaining how to scale from a single AI tool into a **Multi-Agent AI Organization**. I might describe myself as the “CEO of my own AI company,” where I manage a hierarchy of specialized sub-agents.

Here is a synopsis of the main points:

1. The AI Org Structure: Orchestrator vs. Workers

- **The “Chief of Staff”:** The main agent (let’s call him “Henry” for future reference) acts as the boss. It runs on a high-intelligence model like **Claude Opus 4.6** to understand complex goals and delegate tasks.
- **Specialized Sub-Agents:** Instead of asking one AI to do everything, you create specialized “employees” for specific roles.
 - **Coding:** Handled by **GPT 5.4** (highly efficient for code).
 - **Research/Writing:** Handled by cheaper or local models like **Kimi** or **Qwen**.
- **The Scaling Strategy:** Don’t build too many agents at once. I recommend starting with **one sub-agent** (like a coder) and mastering that workflow before adding more.

2. Technical Advantages of Sub-Agents

I identify five reasons why a multi-agent system is superior to a single chatbot:

1. **Parallel Work:** The system can run up to five tasks simultaneously.
2. **Cost Efficiency:** You only use expensive models for the “boss” role; worker tasks are offloaded to cheaper models.
3. **Clean Context:** Each sub-agent gets a fresh context window, preventing the “pollution” or confusion that happens when one chat history gets too long.
4. **Failure Protection:** If one sub-agent crashes during a task, the main orchestrator remains active and can attempt to fix it.
5. **Auto-Archive:** Sub-agents shut down when a task is complete, keeping the workspace tidy.

3. Mission Control Visualization

To manage tmy “company,” I uses a custom dashboard called **Mission Control**:

- **Team Screen:** A visual organizational chart that helps the user (and the lead AI agent) see who is responsible for what.
- **Calendar View:** A tracking system for “Cron Jobs” (automated tasks), allowing the user to see exactly when research, coding, or social media updates are scheduled.
- **Office View:** A “for fun” 2D pixel art visualization that shows the agents “moving around” and working in real-time.

4. Advanced Use Case: The 8:00 AM Researcher

- I demonstrates setting up a sub-agent named **Scout**.
- **The Goal:** Scout is programmed to search the web for trending AI news and “vibe coding” updates every morning at 8:00 AM.
- **The Integration:** Tmy research is automatically fed into the Mission Control system to inform the other agents’ work.

5. The Future: Local AI Clusters

- The ultimate goal is **Local AI**. By running worker agents on local hardware (like a Mac Studio or DGX Spark), you eliminate token costs entirely.
- Tmy allows an army of 100+ agents to work 24/7 in total privacy for only the cost of electricity.

Summary: The above is a blueprint for shifting from “talking to AI” to “running an AI workforce,” emphasizing that **delegation** is the key to massive productivity gains.

Recent updates to OpenClaw, an open-source AI agent tool.

New OpenClaw Updates

- **Full ClawHub Integration:** OpenClaw now includes a full user interface (UI) for “ClawHub,” a repository of skills. Users can browse, search for, analyze, and install community-made or official skills directly within the OpenClaw “Mission Control” dashboard.
- **/btw Command:** Tmy new command allows users to start “side conversations” with the AI that do not get stored in the main session’s context. Tmy is useful for asking quick, unrelated questions without bloating the primary conversation or using excessive tokens.
- **Adjustable Thinking for Subagents:** Users can now customize the “thinking level” (e.g., low, medium, or high) and specific models used for subagents. Tmy allows for more cost-effective and efficient workflows, such as using a smaller, cheaper model for simple automated tasks like web scraping.

Hardware and Model Recommendations

- **Desktop vs. Laptop for AI:** I recommend a desktop setup (like a Mac Studio) over a high-end laptop (like a MacBook Pro) for a serious “AI R&D lab.” I contend that laptops carry a price premium for portability that don’t benefit stationary AI research and can have thermal limitations.
- **Model Preferences:** I currently prefer **Claude Opus** for complex orchestration and task completion, noting that while **GPT-4** is fast and smart, it often fails to “finish” tasks in OpenClaw. However, I suggest using cheaper models (like GPT-4.5 mini or Sonnet) for simpler subagent tasks to save money.

Operational Tips and AI Philosophy

- **Managing Session Bloat:** There is a common issue where automated tasks (cron jobs) create numerous session records that bloat the AI's context window, slowing it down and increasing costs. I recommend periodically deleting these unnecessary sessions to maintain performance.
- **Open Source vs. Closed Source:** I advocate for open-source tools like OpenClaw over closed systems like "Claude Code" because open tools have fewer "guardrails" and can be hacked or customized to perform almost any digital task autonomously.
- **Vibe Coding Strategy:** my approach to content creation and tool usage is to focus on what I'm genuinely "obsessed" with. I believe the only way to be successful is to find a powerful tool, master it, and talk about it authentically rather than copying popular trends.
- **The Future of Local AI:** While I currently use a hybrid approach (cloud for brains, local for specific tasks), I predict that within 6–12 months, local hardware and open-source models will be powerful enough to handle most workflows privately and for free.

About Codex vs Claude Code

Claude Code and **Codex** are two very different stages in the evolution of AI-assisted programming. While both are designed to help with software development, they differ in their fundamental nature, their capabilities, and who created them.

Here is the breakdown of the differences between the two.

1. Fundamental Nature: Agent vs. Model

- **Claude Code (Anthropic):** This is an **agentic tool** (or a "developer agent"). It is a Command Line Interface (CLI) tool that uses a model (Claude 3.5 Sonnet) to interact with your file system, run your terminal, execute tests, and perform git operations. It doesn't just "write" code; it "does" work.
- **Codex (OpenAI):** This is a **foundational LLM** (Large Language Model). It was a specialized version of GPT-3 fine-tuned specifically on public code from GitHub. It was the "brain" that powered the original GitHub Copilot. It is an engine that predicts the next line of code, but it cannot "act" on its own.

2. Capabilities and Autonomy

- **Claude Code:** It has a high degree of autonomy. You can give it a high-level task like *"Find the bug in the login flow and fix the failing test,"* and it will search the codebase, edit files, run the build command, see the error, and iterate until the test passes. It can also explain complex architectures and manage your git commits.
- **Codex:** It was primarily used for **autocomplete and snippet generation**. It was excellent at taking a comment (e.g., `// function to fetch weather data`) and generating the boilerplate code. However, it could not run your compiler, check for linting errors, or understand the "state" of your entire local repository unless you manually pasted it in.

3. Interface (How you use it)

- **Claude Code:** You use it in your **Terminal**. You type `claude` and then enter a natural language prompt. It lives alongside your compiler and your debugger.
- **Codex:** It was accessed via **API** or integrated into **IDEs** (like VS Code via Copilot). Developers rarely interacted with the "Codex" model directly; they interacted with the tools built on top of it.

4. Context and Intelligence

- **Claude Code:** Powered by **Claude 3.5 Sonnet**, which is currently considered one of the best models in the world for coding. It benefits from a massive 200,000-token context window, allowing it to “read” large portions of your project at once.
- **Codex:** A **legacy model**. When it was released in 2021, it was revolutionary. However, by modern standards, its reasoning is limited. Its context window was significantly smaller (around 4,000 to 8,000 tokens), meaning it often “forgot” what was happening in other files.

5. Current Status

- **Claude Code:** This is **cutting-edge**. It was released in early 2025 (Beta) and represents the current trend toward “AI Software Engineers.”
- **Codex:** It is essentially **legacy/deprecated**. OpenAI officially retired the specific Codex models in early 2023, encouraging developers to move to the much more capable GPT-3.5 Turbo and GPT-4 models.

Summary Table

Feature	Claude Code	OpenAI Codex
Developer	Anthropic	OpenAI
Category	Agentic CLI Tool	Foundational Model
Primary Use	Task execution (Fixing, Testing, Refactoring)	Code completion and Autocomplete
Interface	Terminal (CLI)	Code Editor (IDE) integration
Can it run code?	Yes, it uses your local shell	No, it only generates text
Context Window	~200,000 tokens	~4,000 - 8,000 tokens
Status	Active / Beta (Latest)	Retired / Superseded by GPT-4

Summary: Codex is the ancestor of modern AI coding; it showed that AI could write code. Claude Code is the modern successor; it shows that AI can be a functioning teammate that executes tasks.